Dungeon Master console

A Tauri operator client for live scenario injection into Korea. IL-2 Series multiplayer

Field	Value
Document	Technical specification, v0.1 draft
Date	2026-08-19
Target	Korea. IL-2 Series (1CGS), early access
Status	Pre-implementation. Section 11 lists blocking unknowns.

This specification describes an operator console that lets a human game master trigger pre-authored scenarios inside a running IL-2 multiplayer mission, observe the resulting world state, and coordinate with other operators. It covers the desktop client only. The mission-side content it drives is described as a contract, not as a deliverable.

1. Problem and approach

IL-2 missions are static once compiled. The engine provides no runtime object instantiation from an external process. Anything the operator can summon must already exist in the compiled mission file, placed and deactivated at load time.

The console is therefore not a spawner. It is a remote trigger board. It sends named signals over the dedicated server's remote console socket; the mission's own internal logic decides what those signals do. The creative work lives in the mission file. The console's job is to make firing those signals fast, legible, reversible where possible, and safe under pressure.

Design principles

- **The operator is mid-session and distracted.** Every control must be reachable in one click from the default view. No nested menus.
- **Nothing fires by accident.** Destructive or irreversible actions require a deliberate second gesture. Everything else fires immediately.
- **State is observed, never assumed.** The console displays what the server logs report, not what the console believes it sent.
- **The mission is the source of truth for content.** The console reads a manifest authored alongside the mission and renders whatever it finds. Adding a scenario must not require recompiling the app.
- **Degrade honestly.** A dropped socket or a stalled log tail is shown as such, not hidden behind stale data.

2. Scope

In scope	Out of scope
Desktop client (Windows primary, Linux secondary)	The mission file itself and its internal trigger logic

In scope	Out of scope
Remote console TCP client and command dispatch	Mission compilation or resaving
Mission log tailing, parsing, and state projection	Server installation, provisioning, or process supervision
Scenario manifest loading and validation	Player-facing UI, in-game overlays, or client-side mods
Operator audit trail	Persistent campaign state across server restarts
Multi-operator awareness (read-only in v1)	Automated or AI-driven scenario direction

3. Architecture

A Tauri v2 application. All privileged work lives in the Rust core: the webview never opens a socket, never touches the filesystem, and never holds a credential. The frontend is a rendering and input layer that communicates with the core exclusively through Tauri commands (frontend to core, request/response) and Tauri events (core to frontend, push).

This split is deliberate rather than incidental. The remote console session is long-lived and stateful; owning it in Rust means a frontend reload, a route change, or a renderer crash does not drop the connection or lose queued commands.

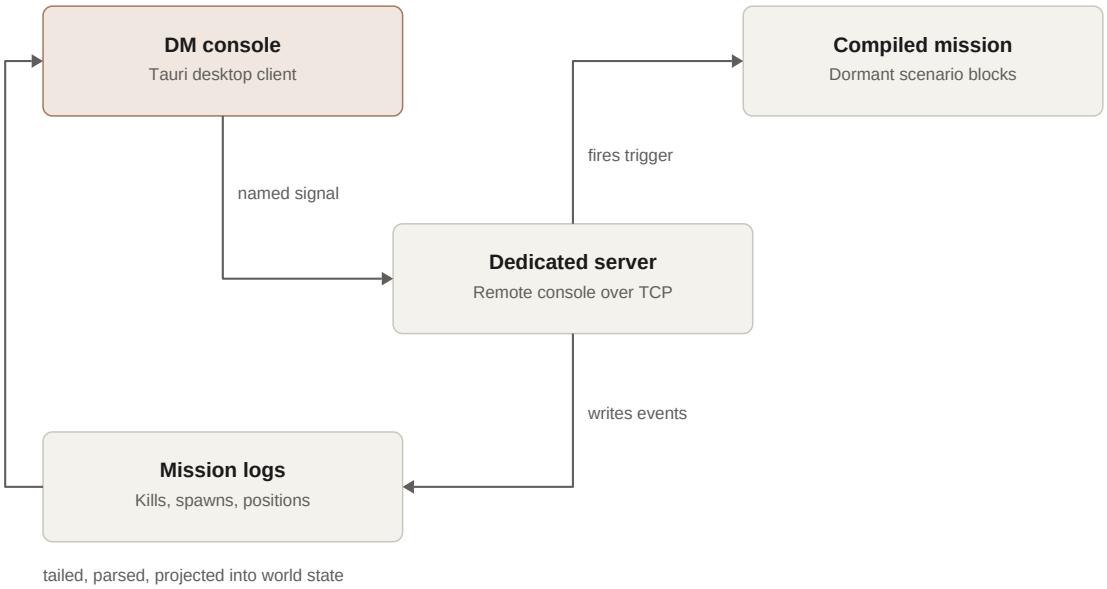

Figure 1. *The signal loop. Commands travel out over the remote console socket; the only return path is the log file. The console has no direct read access to mission state and never will, which is why every design decision downstream treats world state as an inference rather than a fact.*

3.1 Process layout

Layer	Technology	Responsibility
Core	Rust, tokio async runtime	Socket lifecycle, command queue, log tailer, state projection, manifest validation, audit persistence
Bridge	Tauri v2 IPC	Typed commands and events. Schema shared with the frontend via generated TypeScript bindings
Frontend	TypeScript, React, Vite	Scenario board, live state panels, event feed, operator input
Persistence	SQLite via sqlx, in app data dir	Audit log, session history, manifest cache, operator preferences

3.2 Core modules

Module	Description
`console`	Owns the TCP connection to the dedicated server. Handles connect, authenticate, keepalive, backoff reconnect, and framing of the line-oriented protocol. Exposes an async send that resolves when the server acknowledges or times out.
`dispatch`	Serialises outbound commands. Applies per-action cooldowns, global rate limiting, and the confirmation gate. Every dispatch produces an audit record before the command reaches the socket.
`tailer`	Watches the server log directory. Detects rotation, resumes from a stored byte offset, and emits raw log lines. Uses filesystem notification with a polling fallback, since the log directory may be on a network share.
`parser`	Converts raw log lines into typed domain events. Isolated behind a trait so the log grammar can be swapped without touching state logic.
`world`	Projects the event stream into current state: connected players, active scenario instances, recent kills, objective status. Pure reducer over events, which makes it testable against captured log fixtures.
`manifest`	Loads, validates, and hot-reloads the scenario manifest. Validation failures surface as actionable errors naming the offending entry.
`audit`	Append-only record of every operator action with timestamp, operator identity, action id, and server response.

3.3 Failure model

The console must remain useful when the server is partially reachable. Three independent health states are tracked and surfaced separately in the UI, because they fail independently and mean different things:

State	Meaning when unhealthy	Console behaviour
Command channel	Socket closed or auth rejected	Scenario board disabled with a visible banner. Reconnect attempts with exponential backoff, ceiling 30 s. Manual retry always available.
Log channel	Tailer stalled, file missing, or parse failure rate high	World state panels dim and show a staleness age. Commands remain enabled, since firing blind is sometimes correct.
Manifest	File missing or failed validation	Last known good manifest stays loaded. The reload control shows the validation error.

A command that is dispatched while the socket is healthy but receives no acknowledgement within the timeout is marked *unconfirmed*, not *failed*. The distinction matters: the trigger may well have fired. The operator decides whether to retry.

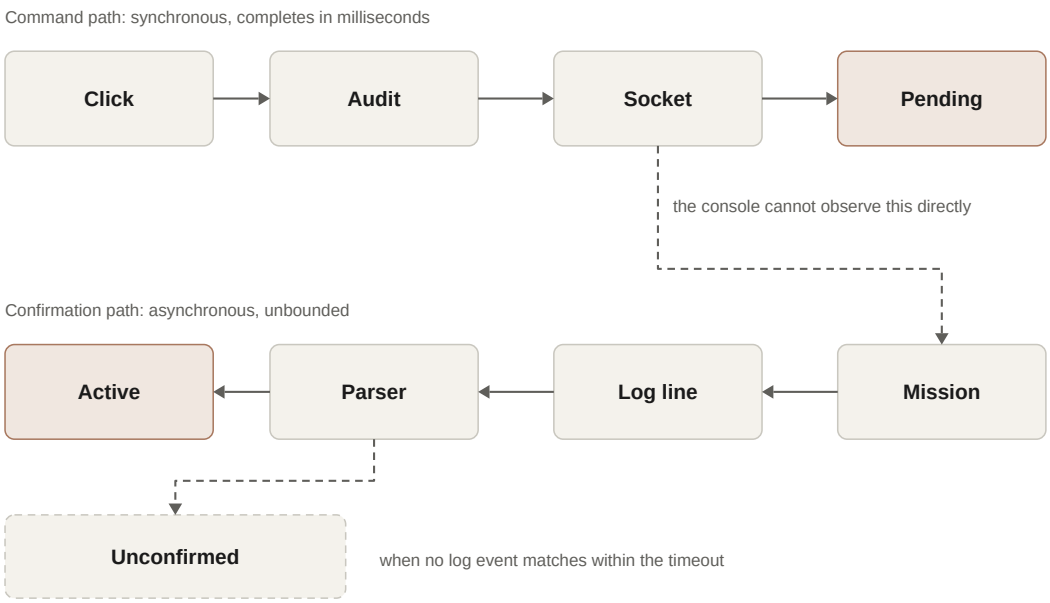

Figure 2. *One dispatch, two independent paths. The command path completes in milliseconds and can only ever tell you the server accepted the string. Whether the mission acted on it is knowable only through the log channel, on an unbounded delay. The dashed segment is the gap the console cannot see across.*

4. Mission-side contract

The console is useless against an arbitrary mission. It requires the mission author to follow a naming and structure convention. This section defines that convention as an interface; the console validates against it and refuses to load a manifest that violates it.

4.1 Signal naming

Every operator-triggerable entry point in the mission is a server input object whose name follows a fixed grammar. Uppercase, underscore separated, three segments minimum:

```
<VERB>_<SUBJECT>[_<QUALIFIER>...]

SPAWN_MIG_SWEEP_NORTH
SPAWN_MIG_SWEEP_NORTH_HEAVY
DESPAWN_MIG_SWEEP_NORTH
ARM_FLAK_SINUIJU
SAFE_FLAK_SINUIJU
SET_ZONE_ALPHA
WEATHER_DETERIORATE
```

The verb is what makes the board legible under load. SPAWN and DESPAWN pair. ARM and SAFE pair. SET applies to dispatch state rather than entities. A scenario that offers no reversal declares that explicitly in the manifest rather than silently omitting a DESPAWN signal.

4.2 Parameterisation by dispatch

The remote console protocol carries a name and nothing else. There is no way to pass coordinates, counts, or timings. Naive expansion of every combination produces a combinatorial explosion of signal names and an

unusable board.

The convention instead splits selection from execution. The operator sets dispatch state with one signal, then executes with another. Mission-side counters and flip-flops route the execution according to the current state. The console models this as a two-stage action and renders the current dispatch state prominently, since a stale zone selection is a likely source of operator error.

```
SET_ZONE_ALPHA      -> mission sets internal zone register to Alpha
SET_SIZE_FLIGHT     -> mission sets internal size register to 4 aircraft
SPAWN_MIG_SWEEP     -> mission reads both registers, activates the
                       matching pre-placed group
```

4.3 Budget constraints

Dormant groups are cheaper than active ones but are not free. They contribute to mission file size, server load time, and per-tick overhead. The manifest must therefore declare a cost weight per scenario, and the console tracks cumulative active weight and warns before the operator exceeds a configured ceiling.

The ceiling value is server-specific and must be established empirically. The console ships with no default; the manifest supplies it. [SEE SECTION 11: object budget calibration]

5. Scenario manifest

A JSON document authored alongside the mission and shipped with it. The console loads it from a user-chosen path and hot-reloads on change. It is the only thing that determines what appears on the board.

```
{
  "schema": 1,
  "mission": "korea_dynamic_v3",
  "budget_ceiling": 0,
  "groups": [
    {
      "id": "air_threats",
      "label": "Air threats",
      "scenarios": ["mig_sweep", "mig_intercept"]
    }
  ],
  "dispatch_registers": [
    {
      "id": "zone",
      "label": "Target zone",
      "options": [
        { "label": "Alpha", "signal": "SET_ZONE_ALPHA" },
        { "label": "Bravo", "signal": "SET_ZONE_BRAVO" }
      ]
    }
  ],
  "scenarios": [
    {
      "id": "mig_sweep",
      "label": "MiG sweep",
      "description": "Fighter sweep from the north, engages nearest",
      "fire": "SPAWN_MIG_SWEEP",
      "reverse": "DESPAWN_MIG_SWEEP",
      "requires": ["zone", "size"],
      "cooldown_s": 120,
      "confirm": false,
```

```
      "weight": 8,
      "expect_log": "spawn_group:MiGSweep"
    }
  ]
}
```

5.1 Field semantics

Field	Notes
`fire`	Signal name sent on activation. Must match the section 4.1 grammar. Required.
`reverse`	Signal that undoes the scenario. Optional. Its absence renders the card as one-way and is shown as such.
`requires`	Dispatch register ids that must hold a value before this scenario can fire. The card is disabled until satisfied, with the missing register named.
`cooldown_s`	Minimum interval between fires, enforced in the core, not the UI. Prevents a double-click from producing a double wave.
`confirm`	When true, the card requires a second deliberate click within a short window. Reserve for scenarios that materially change the session.
`weight`	Contribution to the active object budget. Unitless, calibrated per server.
`expect_log`	A pattern the parser should observe shortly after firing. Its absence within a timeout marks the action unconfirmed. Optional but strongly recommended, since it is the only closed-loop confirmation available.
`budget_ceiling`	Server-specific maximum cumulative weight. A value of 0 disables budget tracking.

Validation runs on load and on every hot reload. It rejects: duplicate scenario ids, references to undefined registers or groups, signal names violating the grammar, and reverse signals that do not pair with a declared fire signal. Errors name the offending entry by id.

6. IPC surface

Commands are frontend-initiated and return a typed result. Events are core-initiated and push to all frontend listeners. Types are defined once in Rust and exported to TypeScript at build time so the two sides cannot drift.

6.1 Commands

Command	Arguments	Returns
`connect`	profile id	Session handle or a typed connection error
`disconnect`	none	Acknowledgement
`fire_scenario`	scenario id, confirmation token if required	Dispatch id for correlating with later events
`reverse_scenario`	scenario id	Dispatch id
`set_register`	register id, option index	Dispatch id
`raw_command`	command string	Dispatch id. Gated behind an explicit expert mode toggle
`load_manifest`	path	Parsed manifest or a structured validation error list
`get_world`	none	Current world state snapshot
`get_audit`	time range, optional filter	Audit records
`broadcast`	message text	Dispatch id

6.2 Events

Event	Payload
`health:changed`	Per-channel health for command, log, and manifest channels, with reason strings
`dispatch:result`	Dispatch id, outcome as confirmed, unconfirmed, rejected, or rate-limited, plus the server response line
`world:patch`	Incremental world state update. Full snapshots are available on request rather than pushed
`log:event`	A single parsed domain event, for the live feed
`log:unparsed`	A raw line the parser did not recognise. Surfaced in a diagnostics pane rather than discarded, since an unrecognised line is usually a grammar change
`manifest:reloaded`	New manifest or validation errors

6.3 Security posture

- The remote console credential is stored in the OS keychain via the Tauri stronghold or keyring plugin. It is never written to the manifest, never logged, and never crosses the IPC boundary to the webview.

- Tauri v2 capabilities are scoped to the minimum: no arbitrary filesystem access from the webview, no shell plugin, no HTTP plugin.
- The Content Security Policy forbids remote script and connect sources. The frontend has no network access of its own.
- The *raw_command* escape hatch is disabled by default and its use is recorded in the audit log with the full command string.

7. Interface

A single view. The operator should never navigate away from the board during a session; anything that would require a second screen belongs in a drawer that overlays without replacing.

7.1 Layout

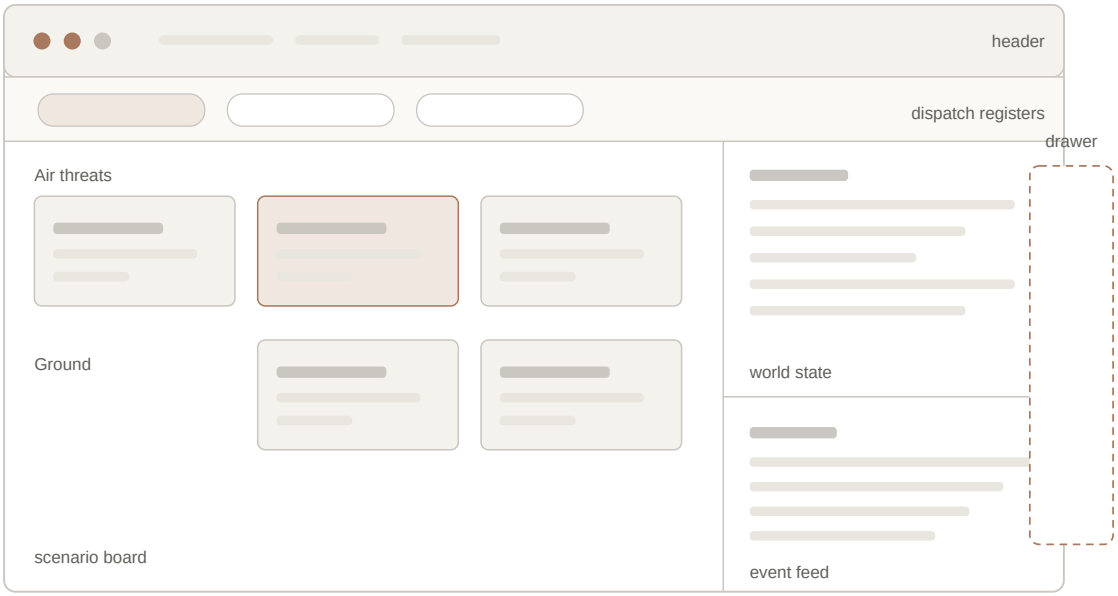

Figure 3. *Region allocation. The scenario board takes roughly two thirds of the width because it is the only surface the operator actually acts on; everything else is read-only context. Drawers slide over the right column rather than replacing the board.*

Region	Content	Behaviour
Header strip	Three health indicators, connected player count, session elapsed time, active object budget	Always visible. The health indicators are the first thing an operator checks when something feels wrong
Dispatch bar	Current value of every dispatch register as a segmented control	Sits directly above the board so a stale selection is hard to miss. Changing a register is itself a dispatched command and shows pending state until confirmed
Scenario board	Cards grouped by manifest group	The primary surface. Each card shows label, cooldown remaining, active instance count, and a reverse control when one exists

Region	Content	Behaviour
World panel	Connected players by side, active scenario instances, recent kills	Right column. Dims with a staleness age when the log channel is unhealthy
Event feed	Chronological parsed events, filterable	Bottom of the right column. Scrolls independently and pins to the newest entry unless the operator has scrolled up
Drawers	Audit log, diagnostics, manifest errors, settings	Overlay from the edge. Dismiss returns to the board with no state lost

7.2 Card states

A scenario card is a small state machine and each state must be visually distinct at a glance, not merely on inspection:

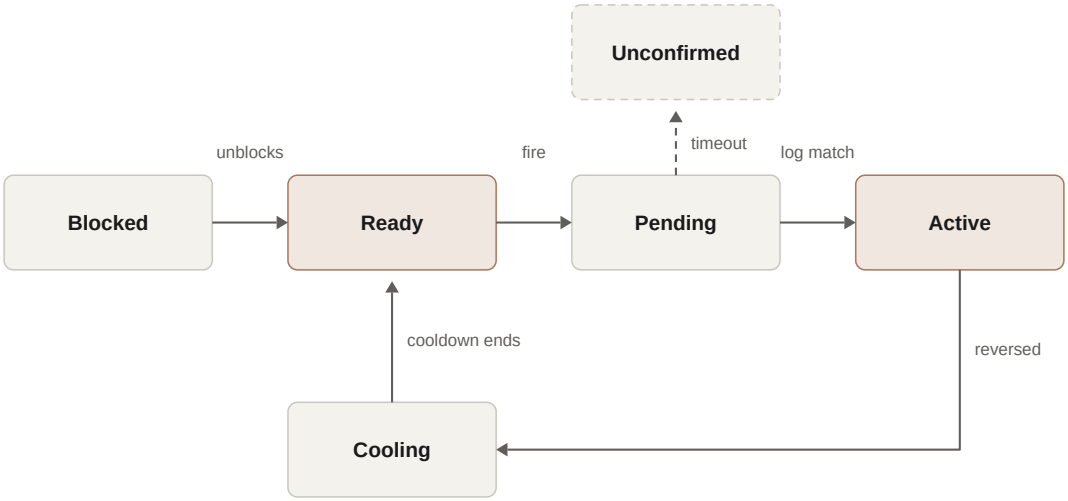

Figure 4. *Card state transitions. Note that unconfirmed is a terminal branch off pending, not a failure state: it is reached by the absence of evidence rather than by any observed event, and both fire and reverse stay available from it.*

State	Trigger	Presentation
Ready	All requirements met, no cooldown	Full contrast, clickable
Blocked	A required register is unset	Reduced contrast with the missing register named on the card
Cooling	Fired within the cooldown window	Countdown displayed. Click is a no-op rather than a queued action
Pending	Dispatched, awaiting confirmation	Indeterminate progress indicator, click disabled
Active	Confirmed and running	Marked active with instance count. Reverse control becomes prominent
Unconfirmed	Timeout elapsed with no expected log event	Warning treatment. Both fire and reverse remain available, because the operator must be able to act on ambiguity

7.3 Confirmation

Confirmation dialogs are modal and therefore expensive during a live session. Cards flagged *confirm* instead use in-place two-stage activation: the first click arms the card and starts a short visible timeout, the second fires. Clicking anywhere else disarms. This keeps the operator's attention on the board and makes accidental activation require two deliberate gestures in the same place.

8. Multi-operator

Version 1 assumes multiple consoles may connect to the same server concurrently and does not attempt to arbitrate between them. It does provide awareness: because every operator action produces an observable effect in the mission logs, each console can detect scenario activations it did not itself initiate and mark those cards active with an *external* attribution.

Genuine coordination, meaning locking, turn-taking, or a shared authority model, requires a coordination service the consoles talk to rather than peer detection. That is deferred. The v1 posture is that operators coordinate out of band, over voice, and the console merely avoids lying to them about the world.

9. Non-functional requirements

Requirement	Target
Time from click to socket write	Under 50 ms at the 95th percentile
Log event to UI reflection	Under 500 ms at the 95th percentile
Sustained log parse throughput	Comfortably above the peak line rate of a full server. Measure before fixing a number. [SEE SECTION 11]
Reconnect after socket loss	First attempt within 1 s, backoff to a 30 s ceiling, indefinite retry
Idle resource use	Low enough to run alongside the sim on the same machine. Tauri's system webview makes this achievable where Electron would not
Cold start to usable board	Under 3 s including manifest load
Audit durability	Every dispatch is persisted before the socket write, not after

10. Build sequence

Ordered so that the riskiest unknowns are resolved before any UI work begins. If milestone 0 fails, the entire approach is invalid and no amount of frontend polish recovers it.

Milestone	Deliverable	Exit criterion
0. Spike	Manual telnet session against a private server running a two-object test mission	One server input fires by hand and the object demonstrably activates in-game. This is the whole project in miniature
1. Core socket	Rust console module with connect, auth, send, reconnect	Command dispatched programmatically produces the same in-game effect
2. Log loop	Tailer and parser producing typed events from a live session	Closed loop: a fired scenario is confirmed by an observed log event
3. Manifest	Schema, loader, validator, hot reload	Adding a scenario to the mission and manifest requires no code change
4. Shell	Tauri app, IPC surface, health strip, scenario board	A scenario fires from a button and the card reaches active state
5. World state	Reducer, world panel, event feed	Operator can answer who is flying and what is running without leaving the board
6. Hardening	Audit log, cooldowns, confirmation, budget tracking, diagnostics	Survives a real session with real players

11. Unverified assumptions

Korea. IL-2 Series runs on a new engine. The architecture above is derived from how the Great Battles series behaves and every inherited assumption below must be confirmed against Korea before implementation. Items marked blocking invalidate the design if they prove false.

Assumption	Status	How to resolve
A dedicated server for Korea exposes a remote console over TCP	**Blocking**	Start a private server, inspect its startup output for a console service line, attempt a raw socket connection
The console protocol includes a command that fires a named trigger inside the running mission	**Blocking**	Milestone 0 spike. Nothing else proceeds until this is demonstrated
The remote console listen address, port, and authentication scheme	Unknown	Server configuration file and startup log. Do not assume the Great Battles defaults carry over
The mission format provides a server-input equivalent object, and its name in the Korea editor	**Blocking**	Mission editor object palette. Ask in the Mission Making and Mission Editor subforum
Mission logs are written as parseable text with a stable grammar	Unknown	Run a session, inspect the log directory, capture fixtures before writing any parser
Log directory location, rotation policy, and flush latency	Unknown	Empirical. Flush latency directly determines the achievable confirmation timeout

Assumption	Status	How to resolve
Deactivated groups carry acceptable runtime cost at the scale this design assumes	Unknown	Load test with progressively larger dormant populations. Establishes the budget ceiling in section 4.3
Server-side integrity checks do not reject missions built this way	Unknown	Confirm with an authored mission on a public-visible server

Recommended before starting: the Servers and Hosting and Mission Making subforums on forum.il2-series.com are the only current sources for most of the above. The Parallel 38 persistent coop project is attempting adjacent work and is worth contacting directly rather than re-deriving their findings independently.

12. Open design questions

- Should unconfirmed dispatches be automatically retried? Arguments exist both ways and the answer likely depends on measured log flush latency.
- Is a coordination service worth building for v2, or do operators reliably self-organise over voice at the scale these sessions actually run?
- Should the console offer scenario sequencing, meaning timed chains of signals, or does that belong in the mission's own logic where it does not depend on the console staying connected? The latter is architecturally cleaner but far slower to iterate on.
- How should the board handle a mission rotation, where the running mission changes underneath a loaded manifest? Detecting the mismatch is straightforward; deciding what to do about it is not.